



A Webinar introducing the Model Context Protocol

Enabling LLMs to safely access real-world tools and data

Featured Presenters



**Moustapha
Gueye**

*Principal
Architect*

Helping Société Générale navigate the complexity of Cloud Native solutions since 2018



**Wassim
Soualhi**

*Sr Solution
Architect*

An engineer at heart and a solutions architect at Mirantis, has spent over many years designing and building various Telco/IT solutions. His current work gives him the opportunity to assist clients with cutting-edge, innovative solutions.



**Mohamed
Belgaied
Hassine**

*Consulting
Architect*

A pure product of IT Consulting, has been helping customers since 2010 be successful with Data Management and Infrastructure. A Kubernetes practitioner since 2017, an AI enthusiast since 2021. He also talks at conferences about production-ready AI.

Today's Webinar

- Understand **MCP's client-server architecture**
- Learn the **six core primitives** of MCP
- See **MCP in action** with a live demo
- Know how to **start building with MCP tomorrow**



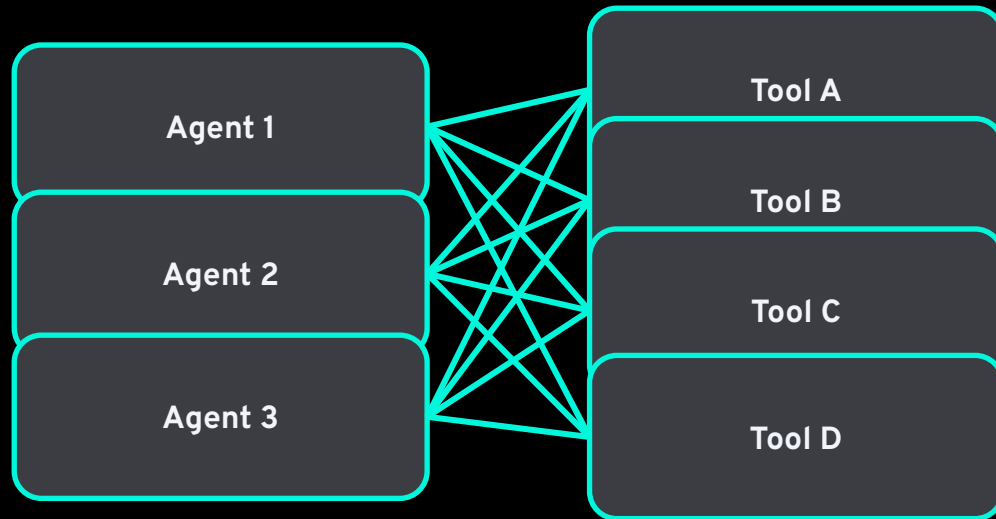
Why do we need MCP

The problem MCP solves, the mental model, and why agents need guardrails

The Problem and Solution

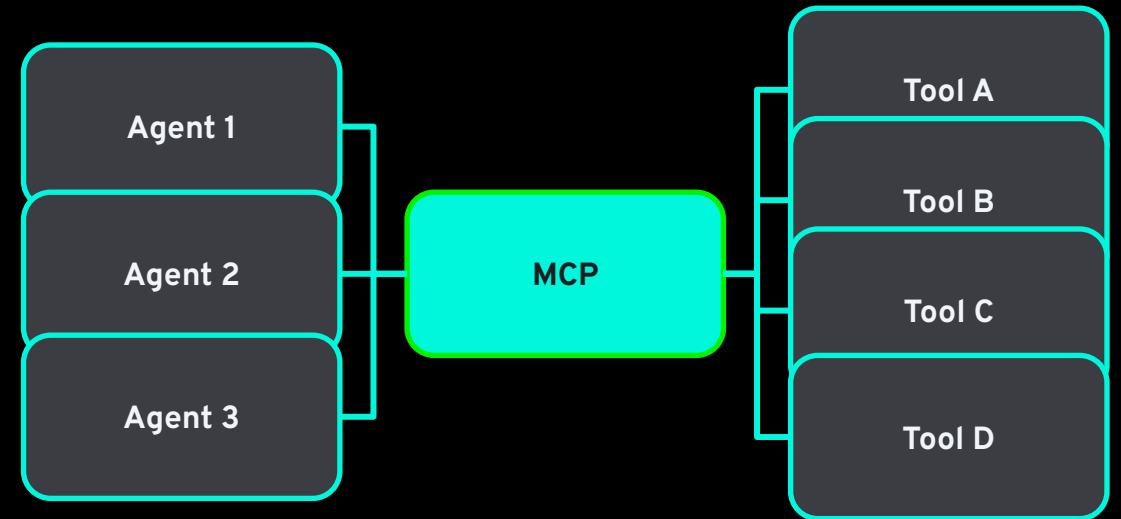
Without MCP:

- Custom integration per tool
- N agents times M tools equals N times M integrations
- Fragile: Breaks when APIs change
- Uncontrolled: No standard access patterns
- Siloed: Each agent reinvents the wheel

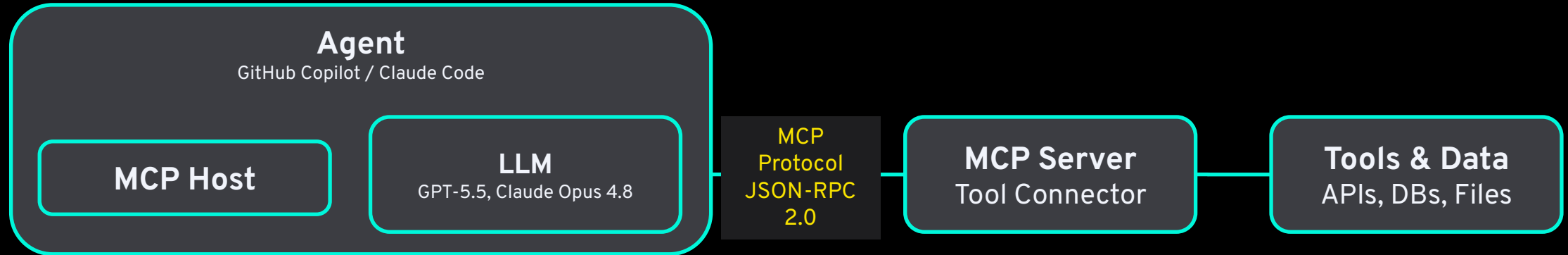


With MCP:

- **Standardized**: One protocol for all tools
- **Pluggable**: Add/swap tools without code changes
- **Composable**: Multiple tools work together
- **Multi-agent ready**: Share servers across agents
- **Controlled**: Discovery-based access



MCP Mental Model



What MCP Is:

- A **standardized protocol** for LLM-to-tool communication
- **Write once**, use across multiple AI hosts
- **Built-in discovery** and capability negotiation
- **Pluggable architecture**: Add/remove servers easily
- **Multi-agent friendly**: Share servers across workflows

What MCP Is NOT:

- **NOT** an agent framework
- **NOT** a hosted service (you run MCP servers)
- **NOT** a replacement for backend services
- **NOT** limited to Anthropic models
- **NOT** a way to expose unlimited tools (context matters)

MCP Tools vs Agent Skills

MCP Tools (Capability):

- Give agents **new abilities** to act
- Access **external systems**: APIs, DBs, files
- Protocol-based, **runtime discovery**
- Shared across agents and hosts
- Example: `get_weather(city)` calls a live API

Agent Skills (Knowledge):

- Teach agents **how to work**
- File-based instructions (**Markdown, YAML**)
- Define workflows, standards, conventions
- Act as **long-term memory** across sessions
- Example: `deploy-checklist.md` defines deployment steps

MCP Tools vs Agent Skills

MCP Tools (Capability):

- Give agents **new abilities** to act
- Access **external systems**: APIs, DBs, files
- Protocol-based, **runtime discovery**
- Shared across agents and hosts
- Example: `get_weather(city)` calls a live API

Agent Skills (Knowledge):

- Teach agents **how to work**
- File-based instructions (**Markdown, YAML**)
- Define workflows, standards, conventions
- Act as **long-term memory** across sessions
- Example: `deploy-checklist.md` defines deployment steps

Together: Skills tell the agent *what to do*. MCP tools let it *actually do it*. A deployment skill orchestrates the workflow; MCP tools execute each step.

MCP Deep Dive

Explaining MCP in more detail

Why LLMs Need Control Mechanisms

Humans Have

- Persistent identity
- Long-term memory
- Accountability
- Human-scale speed

LLMs Lack

- Identity resets each session
- No long-term retention
- Machine-scale speed (1000s API calls per second)
- Probabilistic reasoning

MCP Enables

- Explicit consent: User approval gates
- Structured capabilities: Discovery-based access
- Scoped access: Only what is explicitly exposed

The Six Primitives

Server-side

Tools

Callable Functions
`get_weather(city, units)`

Resources

Readable Data
`resource://weather/docs`

Prompts

Reusable Templates
`weather_advisor(destination)`

Client-side

Sampling

LLM Requests
Server can request completions

Roots

Filesystem boundaries
Scopes the server to a workspace path.

Elicitation

Pause and ask
`weather_advisor()` ← `<missing input>`

Tools Deep Dive

Tool Schema Structure:

- Name: Identifier (e.g., `get_weather`)
- Description: What the tool does
- Input Schema: JSON Schema defining parameters
 - Required versus optional parameters
 - Type validation (string, number, boolean)
 - Enums for constrained choices
 - Default values
- Output Schema
- Example: `get_weather(city, units?, forecast?)`

Why Schemas Matter:

- Agent reasons about **WHEN** to use the tool
- Agent understands **HOW** to invoke it
- User query: "What should I pack for SF?"
 - Agent infers need for weather
 - Agent calls `get_weather("San Francisco")`

Tool Schema Best Practices

Essential Schema Tips:

- **Descriptive names:** Use `verb_noun` pattern (e.g., `get_weather`, not `weather`)
- **Rich descriptions:** Help the LLM understand WHEN to use the tool
- **Constrain inputs:** Use enums, required fields to prevent invalid calls

Example:

```
{
  "name": "get_weather",
  "description": "Get current weather for a city. Use when user asks about weather conditions, temperature, or forecasts.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "city": {"type": "string", "description": "City name"},
      "units": {"type": "string", "enum": ["celsius", "fahrenheit"], "default": "celsius"}
    },
    "required": ["city"]
  }
}
```

Why Multiple Tools Matter

Single Generic Tool:

- `get_weather(city)` returns everything
- Agent must parse large response
- Context bloat: 500+ tokens per call
- Wasted data on unused fields

Multiple Specific Tools:

- `get_temperature(city)` - just temp
- `get_precipitation(city)` - rain/snow
- `get_wind(city)` - wind speed/direction
- `get_forecast(city, days)` - extended

Example: "Should I bike to work?" calls `get_temperature()`, `get_precipitation()`, `get_wind()` - NOT `get_weather()` with 20 unused fields

Tool Count Trade-offs

The Context Window Problem:

Tools Exposed	Schema Tokens	Agent Reasoning
5 tools	~500 tokens	Easy selection
20 tools	~2000 tokens	Moderate
50+ tools	~5000+ tokens	Difficult

Symptoms of Too Many Tools:

- Wrong tool selection:** Agent picks similar-sounding tool
- Context exhaustion:** Less room for conversation
- Slower responses:** More reasoning required
- Hallucinated parameters:** Confusion between schemas

Best Practices:

- Group by domain:** Separate MCP servers per concern
- Progressive exposure:** Start minimal, add as needed
- Clear naming:** Distinct verb_noun patterns
- Rule of thumb:** 5-10 well-designed tools per server

Resources and Prompts

Resources (Readable Data):

- URI: `resource://[server]/[path]`
- Example:
`resource://weather/docs/api-guide`
- Support templates:
`resource://logs/{service}/{date}`

Benefits:

- Context:** Provide background information
- Reference:** Expose lookup data
- Real-time:** Stream live updates

Prompts (Reusable Templates):

- Template with variables
- Example:
`weather_travel_advisor(destination, dates)`

Benefits:

- Consistency:** Standardized behavior
- Expertise:** Domain knowledge built-in
- Reusability:** Same template, different inputs

Tool Invocation Lifecycle

Step-by-Step:

1. **Initialize**: Host sends initialize request, server responds with capabilities
2. **Discovery**: Host learns available tools, resources, and prompts at runtime
3. **Selection**: Agent chooses tool based on user query
4. **Consent**: Host prompts user if configured (client feature)
5. **Invocation**: Host sends tool call to server
6. **Execution**: Server runs function
7. **Response**: Server returns result to host
8. **Display**: Agent incorporates result into response

Dynamic Discovery:

Hosts learn what servers can do at runtime without hardcoding

Long-running tools? MCP's experimental **Tasks** feature lets servers return a task handle immediately and deliver results later

Transports

Transport	Use Case	Pros	Cons
stdio	Claude Desktop, local development	Simple, secure, recommended	Local only
Streamable HTTP	Remote servers, web apps, production	Single endpoint, scalable, resumable	More complex setup
HTTP+SSE (legacy)	Backward compatibility	Widely supported	Deprecated, two endpoints

Choosing a Transport:

Primary: stdio (recommended for Claude Desktop and local use)

Production: Streamable HTTP for remote servers and web applications

Note: HTTP+SSE is deprecated but still supported for backward compatibility

Real-World Workflow: Ticket Triage

User Request:

"Check my open tickets and prioritize the P1s"

Agent Workflow (Multiple Tool Calls):

1. `list_tickets(status="open", assignee="me")`

- Returns 12 tickets

2. `get_ticket("TICK-1234")`

- Gets details on P1 ticket

3. `add_comment("TICK-1234", "Investigating...")`

- Updates ticket

4. `update_ticket_status("TICK-1234", "in_progress")`

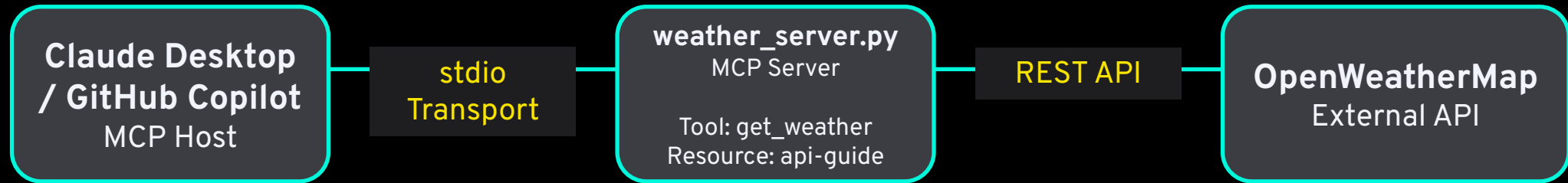
- Changes status

Key Insight: One user request leads to multiple coordinated tool calls. MCP enables the agent to orchestrate complete workflows.

Live Demo

Building a Weather MCP Server

Demo Architecture



What You Will See:

- Server startup managed by Claude Desktop
- Client connecting over stdio transport (recommended)
- Discovery handshake in logs
- Tool invocation with consent (Claude Desktop feature)
- Real API call to OpenWeatherMap
- Error handling for invalid city names

Demo Code Overview

What It Does

- Wraps OpenWeatherMap API
- Defines tool schema
- Handles tool invocations
- Returns structured data
- Provides logging

Key Components

- **Tool Definition:** Describes capability
- **Input Schema:** City + units
- **Handler Function:** API calls
- **Error Handling:** Graceful failures

The Magic: Simple services wrapped in ~50 lines of code

Configuration

Agent Configuration (stdio):

```
{
  "mcpServers": {
    "weather": {
      "command": "python",
      "args": ["-m", "weather_server"],
      "env": {
        "WEATHER_API_KEY": "your-key-here"
      }
    }
  }
}
```

Key Points:

- **stdio**: Simple, secure, recommended by Anthropic for Claude Desktop
- **Server lifecycle**: Managed by client as subprocess
- **API key**: Set in env, not exposed to agent
- **Easy management**: Add or remove servers by editing config

Alternative: Streamable HTTP Transport

Client Configuration (Streamable HTTP) – NOT used in this demo:

```
{
  "mcpServers": {
    "weather-remote": {
      "url": "http://localhost:8000/mcp"
    }
  }
}
```

Use Cases:

- **Remote servers:** Server runs on different machine
- **Web applications:** Browser-based clients
- **Multiple clients:** Share one server across clients

Note: The older HTTP+SSE transport (`/sse` endpoint, `"transport": "sse"`) is deprecated but still supported

Live Demo

Demo Flow:

1. Show server code
2. Start VSCode Copilot
3. Add MCP configuration (stdio option)
4. Test tool call with consent
6. Error handling demo
7. Wrap-up

Demo Debrief: Key Takeaways

- **MCP server** wraps any external service or API
- **Configuration** connects servers to clients
- **User consent** is a client feature (Claude Desktop), not protocol requirement
- **Server logs** show MCP protocol in action
- **Resources** provide structured documentation access
- **Error handling** is built into protocol with structured responses
- **Multiple tools** enable complex workflows (like ticket triage)
- **Tool composition** is where MCP shines
- **Context awareness**: Balance tool count vs. precision
- **Pluggable servers**: Add capabilities without code changes

Security Basics

Core Principles:

- **User consent**: Client feature (e.g., Claude Desktop) that prompts for approval
 - Custom MCP hosts may implement different consent patterns
- **Least privilege**: Only expose what agents need
- **Tool call logging**: Log invocations for debugging and audit

Important Note:

User consent prompts are a CLIENT IMPLEMENTATION feature (e.g., Claude Desktop), NOT a protocol requirement. Custom MCP hosts may implement different consent patterns or auto-approve tools.

Testing Your Server

MCP Inspector - Essential Tool:

Test your server before connecting to Claude Desktop

```
npx @modelcontextprotocol/inspector python -m your_server
```

What Inspector Does:

- **Verify tool schemas:** Check that definitions are correct
- **Test execution:** Invoke tools and see responses
- **Debug issues:** Catch errors before production use
- **Interactive testing:** No client setup required

Your Next Steps

1. **Clone the demo repository** (available via QR code)
2. **Get the Weather server running** with stdio (30 minutes)
3. **Browse the MCP Registry** for pre-built servers
4. **Test with MCP Inspector** before using with Claude Desktop
5. **Build your own MCP server** for a custom use case

Pre-Built Servers to Explore:

- **@modelcontextprotocol/server-filesystem**: File system access
- **@modelcontextprotocol/server-github**: GitHub integration
- **@modelcontextprotocol/server-postgres**: Database access

What You Learned Today

Core Concepts:

- MCP is a **standard protocol** for LLM-to-tool communication
- **Four primitives**: tools, resources, prompts, sampling
- **Tool composition**: Multiple tools enable workflows
- **Context trade-offs**: More tools does not always mean better
- **Pluggable architecture**: Add/swap tools easily
- **stdio transport** is recommended for Claude Desktop
- **Security** via consent (client feature), schemas, and least privilege

Key Insight:

Weather example shows pattern that scales to enterprise use cases (ticket systems, databases, K8s, monitoring, business APIs)

Looking Ahead: MCP and Agentic AI

Today: Narrow Agents

- One agent per task (deploy, triage, ...)
- **Rigid**: Hard-coded workflows
- **Fragile**: Breaks outside its domain
- **Proliferation**: Dozens of agents to manage
- Integration overhead between agents

Tomorrow: General-Purpose Agents

- **One capable agent**, many skills and tools
- **Adaptive**: Skills teach it new domains
- **Extensible**: MCP tools grant new capabilities
- **Composable**: Combine skills and tools dynamically
- One agent, unlimited potential

The shift: Instead of building a new agent for every task, equip one general-purpose agent with **Skills** (knowledge) and **MCP tools** (capabilities) to handle any workflow.

Essential Resources

Official Documentation:

- MCP Specification: spec.modelcontextprotocol.io
- Getting Started: modelcontextprotocol.io
- MCP Registry: <https://registry.modelcontextprotocol.io/>
- MCP Inspector: github.com/modelcontextprotocol/inspector

Official SDKs:

- Python SDK: github.com/modelcontextprotocol/python-sdk
- TypeScript/JavaScript SDK: github.com/modelcontextprotocol/typescript-sdk
- Community SDKs: Go, Rust (varying maturity)

Demo Repository



Demo Repo

github.com/Mirantis-PS/mcp101-training-public



Questions ?



Thank you!

NOW **GO & BUILD**
!